

Universidade Federal do Ceará
Campus Quixadá
QXD0068 – Reuso de Software
Prof. Francisco Victor da Silva Pinheiro

Trabalho 1
Refatoração com Padrões de Projeto em Sistema Legado

Nome: Francisco Josias da Silva Batista

Matricula: 542167

Quixadá-CE
2026

Sumário

1. Introdução
2. Caracterização e Diagnóstico do Sistema Legado
3. Padrões de Projeto Selecionados
4. Refatorações Realizadas

    4.1 Padrão 1 - Facade

    4. 2 Padrão 2 - Strategy

    4.3 Padrão 3 - Factory Method

    4.4 Padrão 4 - State

5. Análise dos Resultados
6. Conclusão
7. Referências

1. Introdução

A evolução de sistemas de software frequentemente acarreta o acúmulo de débito técnico, manifestado por meio de alto acoplamento, baixa coesão e duplicação de código. No contexto de aplicações descentralizadas (dApps) e contratos inteligentes, a refatoração e o reuso de software são ainda mais críticos, visto que o código implantado em blockchain é imutável por padrão e o custo computacional (Gas) é diretamente impactado pela arquitetura do sistema.

O sistema legado selecionado para este trabalho é o Allo Protocol V2, desenvolvido originalmente pela Gitcoin. Trata-se de um protocolo robusto na rede Ethereum (EVM) projetado para alocação e distribuição automatizada e descentralizada de fundos. A motivação para a escolha deste sistema reside na sua relevância no ecossistema Web3 e na oportunidade de analisar como padrões de projeto clássicos da Orientação a Objetos podem ser adaptados para a linguagem Solidity, equilibrando boas práticas de engenharia de software com as limitações inerentes à blockchain.

O objetivo deste trabalho é identificar problemas arquiteturais no núcleo do protocolo (especificamente nos contratos Allo.sol e Registry.sol) e aplicar padrões de projeto do catálogo GoF (Gang of Four) para melhorar a extensibilidade, reduzir o acoplamento e facilitar a manutenção do código.

Repositório do projeto refatorado: https://github.com/josiasdev/allo-v2

2. Caracterização e Diagnóstico do Sistema Legado

2.1 Caracterização do Sistema

O Allo Protocol atua como uma infraestrutura base para que comunidades criem pools de financiamento (por exemplo, financiamento quadrático e doações diretas).

- **Principais funcionalidades:** criação de perfis de governança, instanciação de pools de tesouraria, definição de estratégias de distribuição e repasse de fundos (taxas e alocações).
- **Linguagem e tecnologias:** Solidity (smart contracts) e Foundry (framework de testes e deploy).
- **Estrutura geral:** o projeto orbita em torno de Registry.sol, que gerencia perfis de usuários, e de Allo.sol, que gerencia o capital e as pools. Cada pool utiliza um contrato de estratégia que herda de BaseStrategy.sol.

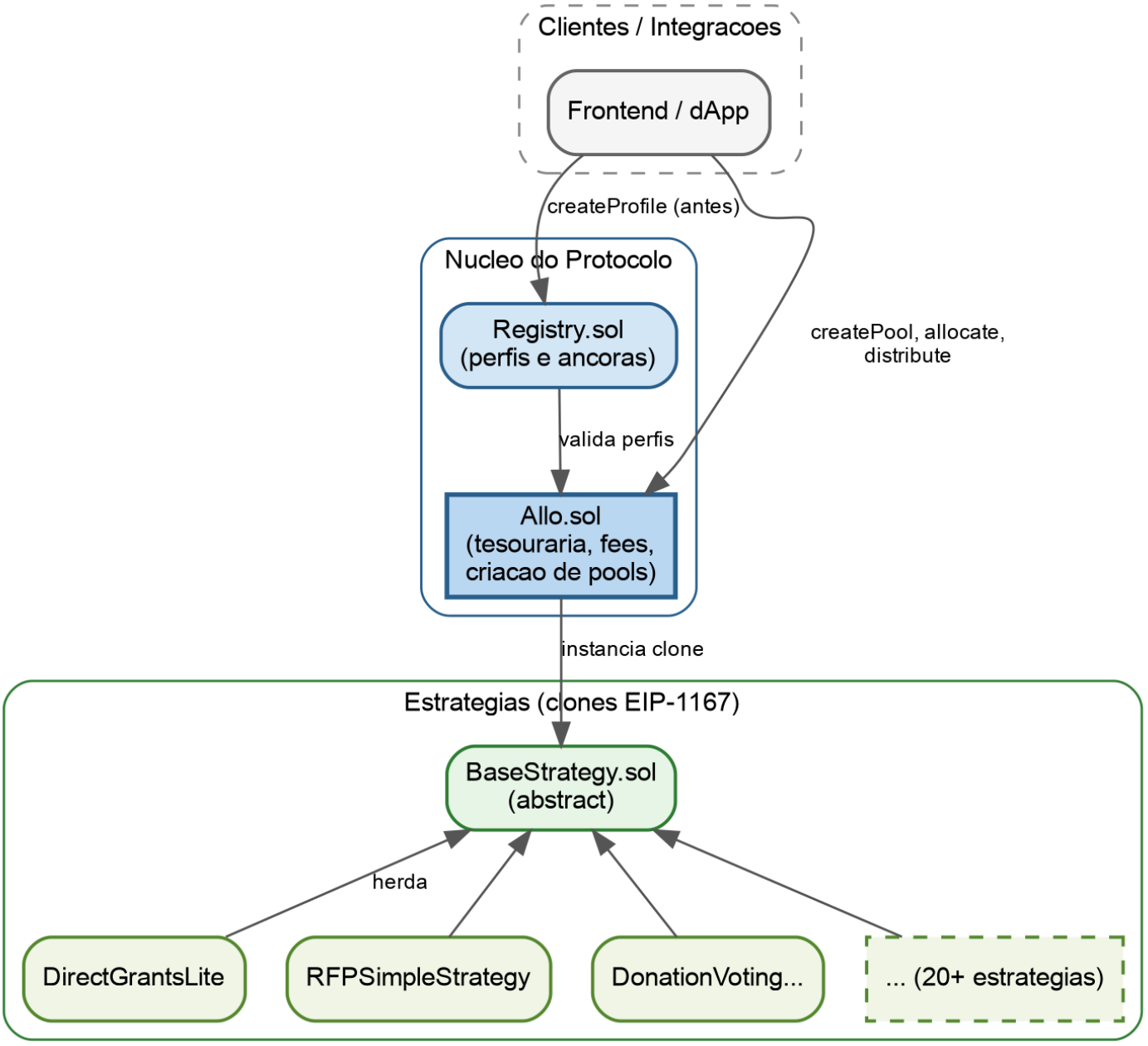


Figura 1: arquitetura do Allo Protocol V2 com Registry.sol, Allo.sol e as estratégias que herdam de

BaseStrategy.sol.

2.2 Problemas Identificados

Apesar de ser um projeto amplamente utilizado, a arquitetura original apresenta desafios estruturais:

- **Responsabilidades excessivas e alto acoplamento (Allo.sol):** o contrato principal atua como uma God Class. Ele gerencia a tesouraria, calcula taxas estáticas, cria clones de contratos de estratégias e valida perfis no Registry.sol.
- **Dificuldade de uso (falta de abstração):** para um desenvolvedor frontend ou sistema externo criar uma campanha, é necessário realizar múltiplas transações sequenciais: primeiro interagir com o Registry para criar um perfil, esperar a confirmação e depois chamar o Allo para criar a pool. Isso encarece o uso e expõe a complexidade do subsistema.
- **Rigidez na lógica de taxas:** a cobrança de taxas possui lógica embarcada no Allo.sol baseada em um percentual fixo. Mudar o modelo de negócios para taxas escalonadas por volume exigiria a reescrita e o redeploy do contrato principal.
- **Excesso de condicionais (code smells):** os contratos que herdam de BaseStrategy.sol dependem de cadeias de require() espalhados para verificar em qual fase a pool está (por exemplo, registro, alocação e distribuição), gerando duplicação de lógica.

A evidência do acoplamento pode ser observada no trecho original da função _fundPool do Allo.sol, em que o cálculo da taxa está inteiramente embarcado no contrato principal:

```
/// @notice Fund a pool.
/// @dev Deducts the fee and transfers the amount to the distribution strategy.
///      Emits a 'PoolFunded' event.
/// @param _amount The amount to transfer
/// @param _poolId The 'poolId' for the pool you are funding
/// @param _strategy The address of the strategy
function _fundPool(uint256 _amount, uint256 _poolId, IStrategy _strategy) internal {
    uint256 feeAmount;
    uint256 amountAfterFee = _amount;

    Pool storage pool = pools[_poolId];
    address _token = pool.token;

    if (percentFee > 0) {
        feeAmount = (_amount * percentFee) / getFeeDenominator();
        amountAfterFee -= feeAmount;

        if (feeAmount + amountAfterFee != _amount) revert INVALID();

        if (_token == NATIVE) {
            _transferAmountFrom(_token, TransferData({from: msg.sender, to: treasury, amount: feeAmount}));
        } else {
            uint256 balanceBeforeFee = _getBalance(_token, treasury);
            _transferAmountFrom(_token, TransferData({from: msg.sender, to: treasury, amount: feeAmount}));
            uint256 balanceAfterFee = _getBalance(_token, treasury);
            // Track actual fee paid to account for fee on ERC20 token transfers
            feeAmount = balanceAfterFee - balanceBeforeFee;
        }
    }

    if (_token == NATIVE) {
        _transferAmountFrom(
            _token, TransferData({from: msg.sender, to: address(_strategy), amount: amountAfterFee})
        );
    } else {
        uint256 balanceBeforeFundingPool = _getBalance(_token, address(_strategy));
        _transferAmountFrom(
            _token, TransferData({from: msg.sender, to: address(_strategy), amount: amountAfterFee})
        );
        uint256 balanceAfterFundingPool = _getBalance(_token, address(_strategy));
        // Track actual fee paid to account for fee on ERC20 token transfers
        amountAfterFee = balanceAfterFundingPool - balanceBeforeFundingPool;
    }

    _strategy.increasePoolAmount(amountAfterFee);

    emit PoolFunded(_poolId, amountAfterFee, feeAmount);
}
```

Trecho 1: lógica de taxas hardcoded no contrato Allo.sol (versão original).

3. Padrões de Projeto Selecionados

Para mitigar os problemas encontrados, foram aplicados quatro padrões GoF:

1. **Facade (Estrutural)**
    - **Problema:** múltiplas chamadas complexas exigidas do cliente para interagir com Registry e Allo.
    - **Justificativa:** prover uma interface unificada e simplificada para a criação de campanhas de financiamento.
    - **Componentes:** criação do contrato AlloStarterFacade.sol.
    - **Resultado esperado:** redução do acoplamento entre o cliente (frontend) e os subsistemas complexos do protocolo.
2. **Strategy (Comportamental)**
    - **Problema:** lógica de cálculo de taxas engessada dentro do Allo.sol.
    - **Justificativa:** encapsular os algoritmos de taxas, permitindo que o modelo econômico mude em tempo de execução sem alterar o contrato base (princípio Open/Closed).
    - **Componentes:** interface IFeeStrategy.sol e implementação FlatFeeStrategy.
    - **Resultado esperado:** extensibilidade imediata do modelo de negócios do protocolo.
3. **Factory Method (Criacional)**
    - **Problema:** o Allo.sol possui a responsabilidade de clonar e inicializar novas estratégias de alocação, inchando seu bytecode.
    - **Justificativa:** delegar a instanciação de objetos (contratos) para uma classe específica, facilitando a adição de novos métodos de deploy no futuro.
    - **Componentes:** criação de StrategyFactory.sol.
    - **Resultado esperado:** redução do tamanho e das responsabilidades do Allo.sol.
4. **State (Comportamental)**
    - **Problema:** validação repetitiva de estado da pool por meio de longas cadeias de condicionais nas estratégias de distribuição.
    - **Justificativa:** isolar as regras de transição e os comportamentos permitidos para cada fase da pool.
    - **Componentes:** contratos de estado modulares (RegistrationOpenState RegistrationClosedState) aplicados à estratégia DirectGrantsLite.
    - **Resultado esperado:** eliminação da duplicação de código de validação do ciclo de vida.

4. Refatorações Realizadas

Nesta seção são descritas as alterações realizadas para cada padrão aplicado. Todas as refatorações preservaram as assinaturas públicas dos contratos e a suíte de testes legada, de modo que o comportamento observável do protocolo permaneceu inalterado.

4.1 Padrão 1 - Facade

- **Situação Original:** O protocolo expõe seus subsistemas (Registry e Allo) de forma fragmentada. Para iniciar uma campanha de doação, um desenvolvedor integrador é forçado a orquestrar múltiplas chamadas complexas, conhecendo os detalhes internos de ambos os contratos e forçando o usuário final a pagar as taxas de Gas de duas transações separadas.
- **Problema identificado:** Alto acoplamento entre o cliente externo e a complexidade do subsistema do Allo Protocol, dificultando a usabilidade e a adoção da ferramenta.
- **Solução implementada:** Criou-se um novo contrato chamado AlloStarterFacade. Ele atua como uma interface única e simplificada que encapsula a criação do perfil no Registry e a inicialização da pool no Allo em uma única transação, exigindo do cliente externo apenas os parâmetros essenciais. A injeção das dependências ocorre no construtor, e o facade delega internamente as chamadas para os contratos reais do protocolo.

Trecho de código (Antes):

Antes da refatoração, o cliente precisava orquestrar duas transações e conhecer as APIs de dois subsistemas distintos:

```
// Codigo do cliente (antes): duas transacoes e dois contratos
bytes32 profileId = registry.createProfile(
    nonce, name, metadata, msg.sender, managers
); // tx 1
uint256 poolId = allo.createPool(
    profileId, strategy, initData,
    token, amount, metadata, managers
); // tx 2
```

Trecho 2: fluxo de criação de campanha antes do Facade.

Trecho de código (depois):

```
19      function initializeCampaign(    undefined gas
20          uint256 _nonce,
21          string memory _name,
22          Metadata memory _metadata,
23          address _strategy,
24          bytes memory _initStrategyData,
25          address _token,
26          uint256 _amount,
27          address[] memory _managers
28      ) external returns (uint256 poolId) {
29
30          // The Facade orchestrates the complexity of the subsystem internally.
31          bytes32 profileId = registry.createProfile(
32              _nonce, _name, _metadata, msg.sender, _managers
33          );
34
35          return allo.createPool(
36              profileId, _strategy, _initStrategyData, _token, _amount, _metadata, _managers
37          );
38      }
39  }
```

Trecho 3: interface unificada do AlloStarterFacade (depois).

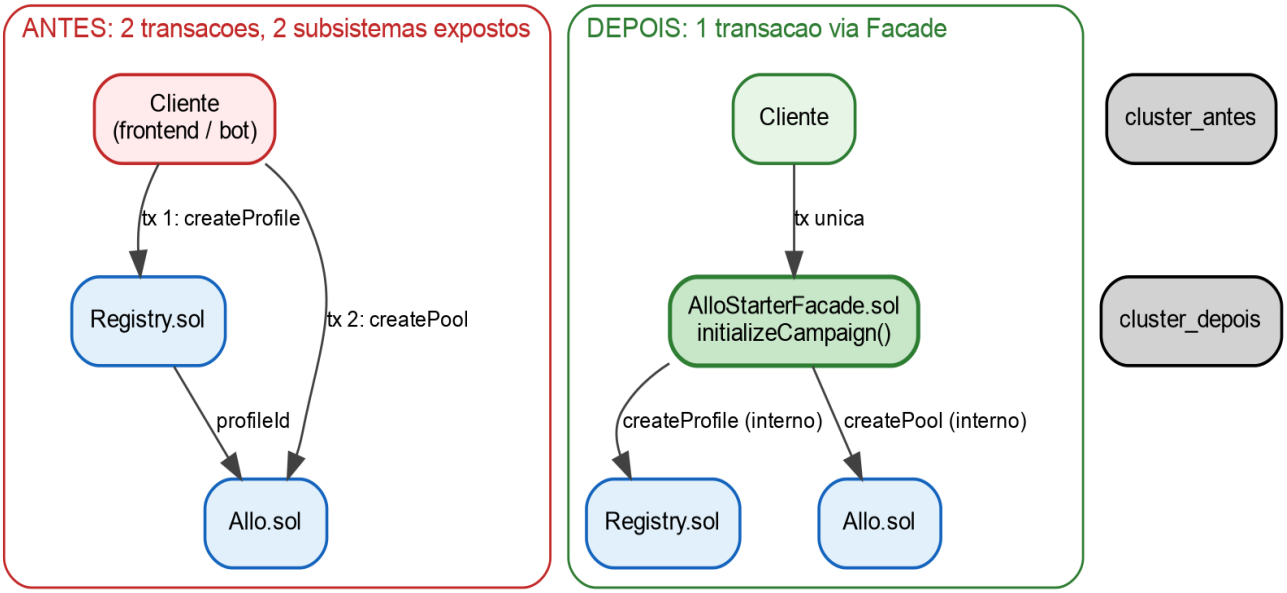


Figura 2: comparação do fluxo de criação de campanha antes (duas transações) e depois (transação única via AlloStarterFacade).

4.2 Padrão 2 - Strategy

- **Situação original:** O cálculo da taxa de financiamento de uma pool estava escrito diretamente no interior da função _fundPool do Allo.sol, condicionado a uma variável de estado percentFee. Qualquer alteração no modelo econômico do protocolo, como a criação de taxas progressivas ou isenções por perfil, exigiria modificar e redeployar o contrato central, que é o coração do protocolo e possui forte restrição de mudança por segurança.

- **Problema identificado:** Violação do princípio Open/Closed: o contrato principal estava aberto para modificação e fechado para extensão. A lógica de negócio referente às taxas estava acoplada à lógica de transferência de valores e ao gerenciamento da pool.
- **Solução implementada:** Adotou-se o padrão Strategy, isolando o algoritmo de cálculo de taxas por trás de uma nova interface, IFeeStrategy, com um único método, calculateFee. Foi criada a implementação concreta FlatFeeStrategy, que reproduz fielmente o cálculo legado (percentual fixo sobre o valor, com denominador de 1e18 correspondente a 100%). O contrato Allo.sol passou a armazenar uma referência injetável à estratégia de taxas e a delegar o cálculo por meio de um setter restrito ao proprietário (updateFeeStrategy), emitindo o evento FeeStrategyUpdated. A função updatePercentFee foi mantida por compatibilidade, recriando a instância de FlatFeeStrategy para sincronizar o valor da taxa.

Trecho de código (antes):

```
// Allo.sol (antes): calculo da taxa embarcado
if (percentFee > 0) {
    feeAmount = (_amount * percentFee) / getFeeDenominator();
    amountAfterFee -= feeAmount;
    // ...
}
```

Trecho 4: cálculo de taxa acoplado ao Allo.sol (antes).

Trecho de código (depois)

```
// IFeeStrategy.sol (nova interface)
interface IFeeStrategy {
    function calculateFee(uint256 _amount) external view returns (uint256);
}

// FlatFeeStrategy.sol (implementacao concreta)
contract FlatFeeStrategy is IFeeStrategy, Errors {
    uint256 public constant FEE_DENOMINATOR = 1e18;
    uint256 public percentFee;

    constructor(uint256 _percentFee) {
        if (_percentFee > FEE_DENOMINATOR) revert INVALID_FEE();
        percentFee = _percentFee;
    }

    function calculateFee(uint256 _amount) external view returns (uint256) {
        return (_amount * percentFee) / FEE_DENOMINATOR;
    }
}

// Allo.sol (depois): delegacao para a estrategia injetada
uint256 feeAmount = feeStrategy.calculateFee(_amount);
if (feeAmount > 0) {
    amountAfterFee -= feeAmount;
    // ... transferencias inalteradas ...
}
```

Trecho 5: interface, implementação concreta e delegação no Allo.sol (depois).

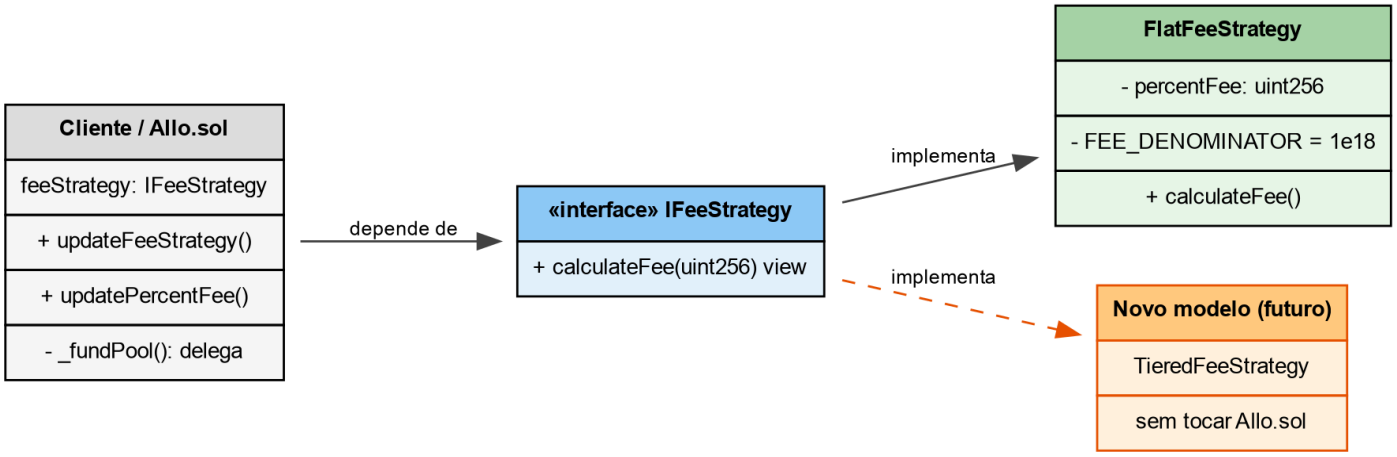


Figura 3: classes do padrão Strategy com a interface IFeeStrategy, o contexto Allo.sol, a implementação FlatFeeStrategy e a extensão futura.

4.3 Padrão 3 - Factory Method

- **Situação original:** A criação de uma pool exigia que o Allo.sol clonasse o contrato de estratégia por meio da biblioteca Clone (padrão minimal proxy EIP-1167) e incrementasse um nonce interno para garantir endereços únicos gerados via CREATE2. Essa responsabilidade de instanciação estava embarcada na função createPoolWithCustomStrategy, misturando a orquestração de negócio com a mecânica de deploy de contratos.
- **Problema identificado:** Acoplamento entre a criação de pools e a técnica de clonagem. O bytecode do Allo.sol crescia com cada responsabilidade adicional de instanciação, e a troca da estratégia de clonagem, por exemplo para um factory com lógica de salt diferente, exigiria alterar o contrato central.
- Solução implementada: Aplicou-se o padrão Factory Method por meio da nova interface IStrategyFactory e da implementação StrategyFactory, que encapsula a lógica de nonce por chamador e de clonagem CREATE2. O Allo.sol recebe a factory por injeção no construtor (padrão inicialização seguido das refatorações anteriores do próprio projeto acadêmico), expõe o setter updateFactory restrito ao proprietário e delega a instanciação em uma única linha. O nonce legado _nonces foi mantido no storage por compatibilidade de layout, porém deixou de ser utilizado.

Trecho de código (antes):

```
// Allo.sol (antes): clonagem inline na criacao da pool
return _createPool(
    _profileId,
    IStrategy(Clone.createClone(_strategy, _nonces[msg.sender]++)),
    _initStrategyData,
    _token,
    _amount,
    _metadata,
    _managers
);
```

Trecho 6: instanciação de clones embarcada no Allo.sol (antes).

Trecho de código (depois):

```
// IStrategyFactory.sol (nova interface)
interface IStrategyFactory {
    function createStrategyClone(address _strategy) external returns (IStrategy);
}

// StrategyFactory.sol (implementacao concreta)
contract StrategyFactory is IStrategyFactory {
    mapping(address => uint256) private _nonces;

    function createStrategyClone(address _strategy) external returns (IStrategy) {
        return IStrategy(Clone.createClone(_strategy, _nonces[msg.sender]++));
    }
}

// Allo.sol (depois): delegacao para a factory injetada
return _createPool(
    _profileId,
    strategyFactory.createStrategyClone(_strategy),
    _initStrategyData,
    _token,
    _amount,
    _metadata,
    _managers
);
```

Trecho 7: interface, factory concreta e delegação no Allo.sol (depois).

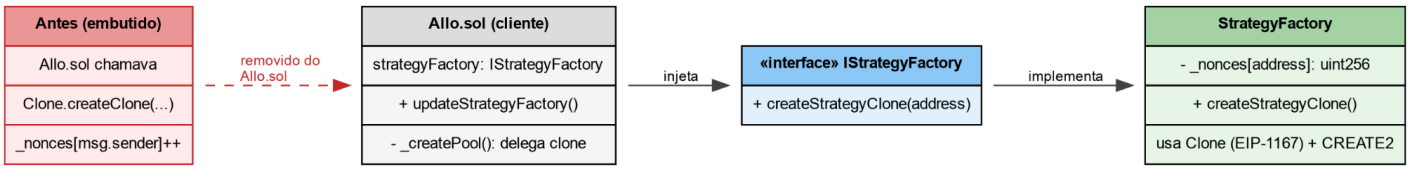


Figura 4: classes do padrão Factory Method com Allo.sol (cliente), IStrategyFactory (produto abstrato) e StrategyFactory (produto concreto).

4.4 Padrão - State

- **Situação original:** A estratégia DirectGrantsLite, uma das implementações concretas de BaseStrategy, controla o ciclo de vida da pool por meio de dois carimbos de tempo (início e fim do registro). A validação dessa janela estava duplicada em duas funções internas: _checkOnlyActiveRegistration, usada pelo modificador onlyActiveRegistration para reverter com REGISTRATION_NOT_ACTIVE, e _isPoolActive, que reapresentava a mesma condição em forma de retorno booleano. O distribuidor de fundos reverte com NOT_IMPLEMENTED de forma fixa, sem participar de nenhuma estrutura de estados.
- **Problema identificado:** Duplicação da regra de negócio do ciclo de vida e ausência de um modelo explícito de fases. A adição de novas fases, como uma janela de alocação ou de distribuição, exigiria replicar a mesma cadeia de condicionais em mais pontos do contrato, aumentando o risco de divergência entre as validações.
- **Solução implementada:** Adotou-se o padrão State com resolução preguiçosa (lazy) do estado atual a partir dos carimbos de tempo, o que dispensa a manutenção de um estado armazenado em storage e torna as transições imunes a atualizações feitas por updatePoolTimestamps. Foi criado o contrato abstrato LifecycleState, que define a interface de validação do ciclo de vida (checkRegistration, checkAllocation, checkDistribution e isPoolActive) e o enum PoolState. Duas implementações concretas e imutáveis foram adicionadas: RegistrationOpenState, em que o registro é permitido, e RegistrationClosedState, em que o registro reverte com REGISTRATION_NOT_ACTIVE. O contrato DirectGrantsLite passou a atuar como Contexto: o método interno _currentState resolve qual objeto de estado está ativo e os pontos de entrada delegam a validação a ele. Os objetos de estado são stateless e são implantados uma única vez no construtor como immutable, sendo compartilhados pelos clones por meio do código da implementação, sem custo adicional de storage.

Trecho de código (antes):

```
// DirectGrantsLite.sol (antes): regra duplicada em dois pontos
function _checkOnlyActiveRegistration() internal view {
    if (registrationStartTime > block.timestamp
        || block.timestamp > registrationEndTime) {
        revert REGISTRATION_NOT_ACTIVE();
    }
}

function _isPoolActive() internal view override returns (bool) {
    if (registrationStartTime <= block.timestamp
        && block.timestamp <= registrationEndTime) {
        return true;
    }
    return false;
}
```

Trecho 8: validação de fase duplicada no DirectGrantsLite (antes).

Trecho de código (depois):

```
// DirectGrantsLite.sol (depois): resolucao do estado e delegacao
function _currentState() internal view returns (LifecycleState) {
    if (registrationStartTime <= block.timestamp
        && block.timestamp <= registrationEndTime) {
        return registrationOpenState;
    }
    return registrationClosedState;
}

modifier onlyActiveRegistration() {
    _currentState().checkRegistration();
    _;
}

// RegistrationClosedState.sol (estado concreto)
function checkRegistration() external pure override {
    revert REGISTRATION_NOT_ACTIVE();
}
```

Trecho 9: delegação das validações para o estado atual (depois).

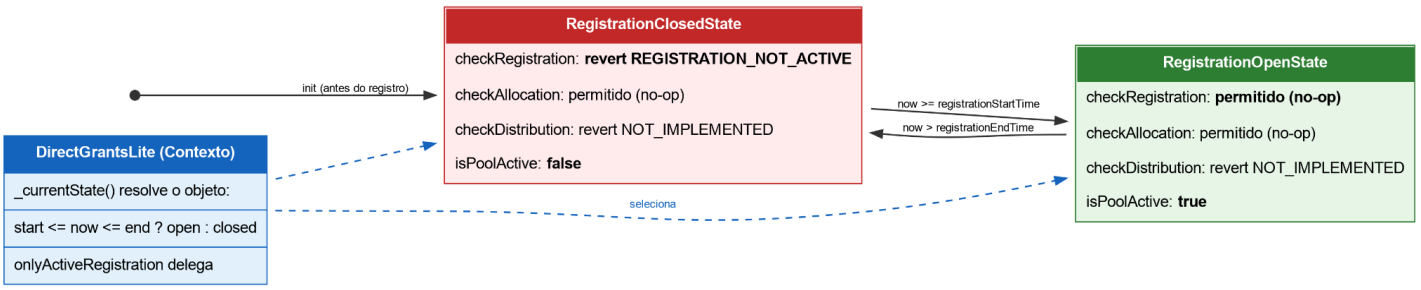


Figura 5: máquina de estados do DirectGrantsLite com as transições RegistrationClosed e RegistrationOpen
baseadas em block.timestamp, registrationStartTime e registrationEndTime.

5. Análise dos Resultados

As refatorações foram conduzidas de forma incremental, com um commit por padrão aplicado, e validadas continuamente por meio da suíte de testes do Foundry. A comparação entre o sistema legado e o sistema refatorado é apresentada a seguir nos principais aspectos de qualidade discutidos na disciplina.

5.1 Redução de acoplamento:

O contrato Allo.sol deixou de conhecer três detalhes de implementação que antes eram fixos em seu corpo: o algoritmo de taxas, a técnica de clonagem de estratégias e as regras de fase das pools. Cada um desses detalhes foi movido para um contrato dedicado (FlatFeeStrategy, StrategyFactory e os estados de ciclo de vida, respectivamente), relacionados ao núcleo apenas por interfaces. O cliente externo, por sua vez, passou a interagir com o subsistema de criação de campanhas por meio do AlloStarterFacade, reduzindo o número de contratos que precisa conhecer.

5.2 Melhoria da coesão:
Cada nova responsabilidade isolada possui um único motivo de mudança. A alteração do modelo econômico toca apenas o pacote de estratégias de taxa; a troca da mecânica de clones toca apenas a factory; a inclusão de novas fases do ciclo de vida toca apenas os contratos de estado. O núcleo do protocolo permanece coeso com suas atribuições originais: gerenciar papéis, tesouraria e o fluxo de funding das pools.

5.3 Redução de duplicação:
No caso do padrão State, a regra da janela de registro estava escrita duas vezes no mesmo contrato (uma para reverter e outra para retornar booleano). Após a refatoração, existe uma única fonte de verdade: o objeto de estado resolvido por _currentState. O mapeamento dos comportamentos por fase ficou explícito em cada classe concreta, o que facilita a revisão pelos auditores de segurança, público-alvo natural de contratos inteligentes.

5.4 Facilidade de manutenção e extensibilidade:

Os quatro padrões aplicados ampliaram o sistema sem modificar o comportamento existente. Exemplos concretos de extensibilidade ganha:

- implementar um modelo de taxas progressivas cria uma nova classe implementando IFeeStrategy e injeta via updateFeeStrategy, sem redeploy do Allo.sol;
- adotar uma estratégia de clonagem distinta, como um factory com controle de allowlist, substitui apenas a instância de IStrategyFactory;
- a estratégia DirectGrantsLite e demais estratégias com janelas de tempo, como as variantes de LTIP e doação com votação, podem reutilizar os mesmos contratos de estado, aproveitando o mesmo ciclo de registro aberto e fechado.

5.5 Reuso de componentes:

- Os componentes criados são reutilizáveis tanto entre estratégias quanto por integrações externas. O AlloStarterFacade pode ser estendido por projetos que queiram expor fluxos simplificados ao usuário final. As estratégias de taxa e a factory são injetáveis e podem ser substituídas por implementações de terceiros que implementem as respectivas interfaces. Os contratos de estado do padrão State são deliberadamente stateless, o que permite compartilhá-los entre múltiplas pools e instâncias de estratégia clonadas.

5.6 Organização, legibilidade e verificação por testes:

- O trabalho introduziu 32 novos testes automatizados além da suíte legada: 11 para o padrão Strategy (FeeStrategy.t.sol), 7 para o Factory Method (StrategyFactory.t.sol) e 14 para o padrão State (DirectGrantsLiteState.t.sol). A suíte completa do projeto apresenta 1.147 testes aprovados, sem nenhuma falha relacionada às refatorações. As três suítes que falham no ambiente de execução dependem exclusivamente da ausência de variáveis de ambiente de RPC para testes de fork (MAINNET_RPC_URL e OPGOERLI_RPC_URL), condição conhecida e pré-existente ao trabalho. Todos os testes legados de cada subsistema foram

mantidos verdes, o que confirma a preservação do comportamento observável, incluindo erros customizados, eventos e assinaturas externas.

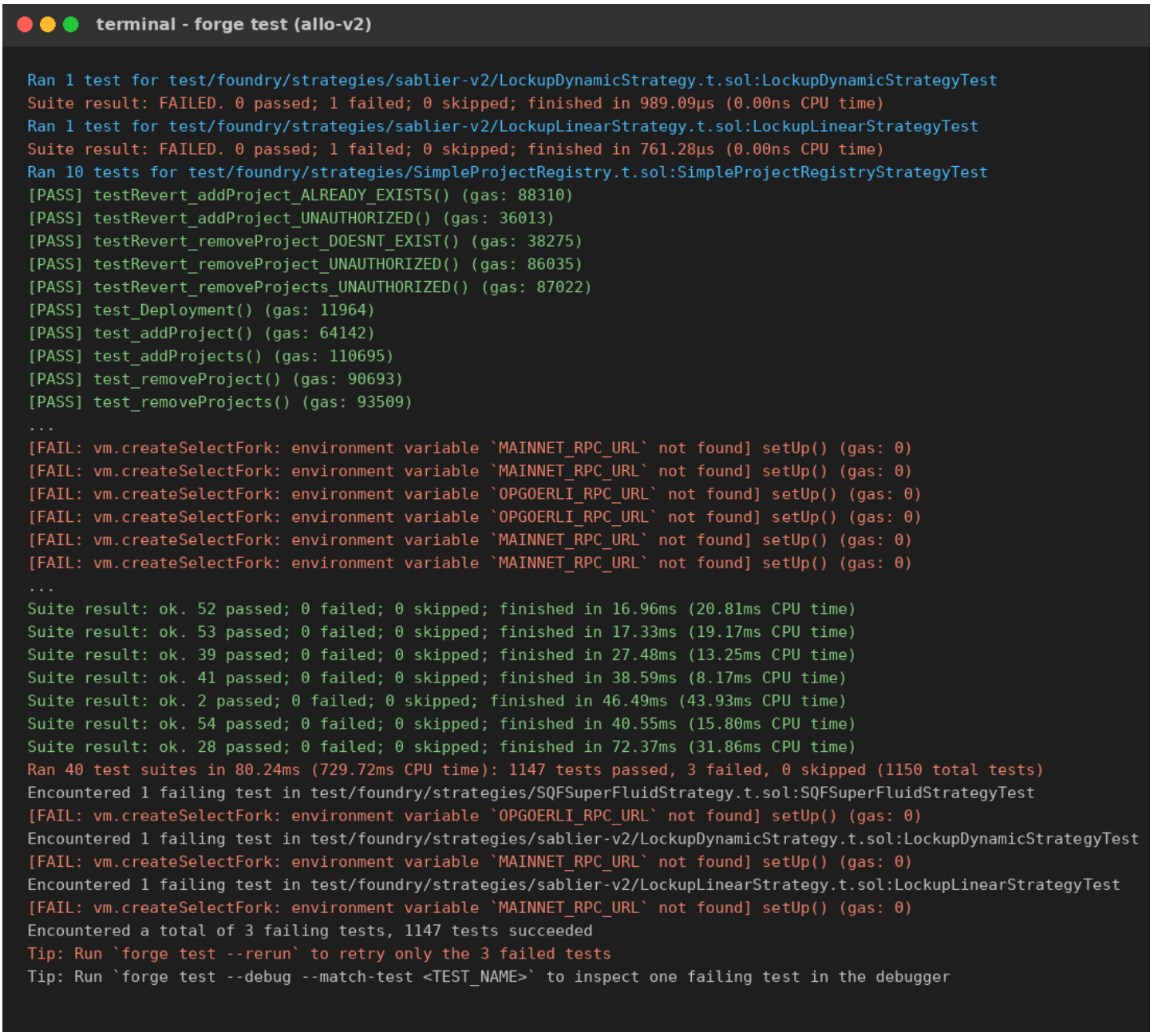


Figura 6: execução da suíte completa com forge test, evidenciando 1.147 testes aprovados e apenas as falhas ambientais de fork por ausência de RPC.

5.7 Impacto no custo de Gas:

As delegações introduzidas pelos padrões Strategy e State acrescentam chamadas externas de baixo custo: após o primeiro acesso a uma conta em uma transação, as chamadas subsequentes a contratos já quentes têm custo marginal reduzido na EVM. Os objetos de estado do padrão State são implantados uma única vez no construtor da implementação e compartilhados pelos clones por meio do mecanismo de immutable, não havendo consumo adicional de storage por pool criada. O

trade-off entre clareza arquitetural e custo de execução foi considerado aceitável, sobretudo em operações de baixa frequência como financiar uma pool ou registrar um beneficiário.

6. Conclusão

Este trabalho aplicou quatro padrões do catálogo GoF ao Allo Protocol V2, sistema legado amplamente utilizado no ecossistema de financiamento descentralizado. O padrão Facade simplificou a criação de campanhas por meio do novo contrato AlloStarterFacade; o padrão Strategy tornou pluggável o cálculo de taxas com a interface IFeeStrategy e a implementação FlatFeeStrategy; o padrão Factory Method extraiu a clonagem de estratégias para a StrategyFactory; e o padrão State eliminou a duplicação das validações de ciclo de vida em DirectGrantsLite por meio dos estados RegistrationOpenState e RegistrationClosedState.

Os benefícios obtidos envolvem menor acoplamento do núcleo do protocolo, maior coesão dos novos módulos, eliminação de duplicação de regras de negócio e ampliação da extensibilidade sem comprometer a compatibilidade com a suíte de testes legada. A capacidade de trocar o modelo econômico de taxas ou a mecânica de clonagem por meio de injeção de dependência ilustra bem o valor do reuso orientado a interfaces no contexto de smart contracts.

As dificuldades encontradas decorrem das particularidades da linguagem Solididade e do modelo de execução da EVM: a necessidade de preservar o layout de storage dos contratos clonados impediu a remoção do nonce legado no Allo.sol; as regras de sobrescrita de funções restringiram a delegação do padrão State a funções view em vez de pure em alguns pontos; e o custo de Gas exigiu que os objetos de estado fossem stateless e compartilhados via immutable. A exigência de manter em verde toda a suíte legada, com mais de mil testes, funcionou como rede de segurança e como critério objetivo de não regressão.

Como aprendizados, destaca-se que padrões de projeto clássicos da engenharia de software são transladáveis para o domínio de contratos inteligentes desde que se respeitem suas restrições de custo e de imutabilidade; que testes automatizados são o principal instrumento para refatorar sistemas legados com confiança; e que a separação de responsabilidades, mesmo em um ambiente hostil a mudanças como uma blockchain, continua sendo um fator determinante para a evolução e o reuso do software.

7. Referências

GAMMA, E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley, 1994.

SOMMERVILLE, I. Engenharia de Software. 10. ed. Pearson, 2019.

BARTOLETTI, M.; POMPIANU, L. An empirical analysis of smart contracts: platforms, applications, and design patterns. In: International Conference on Financial Cryptography and Data Security. Springer, 2017. Disponível em: https://arxiv.org/abs/1703.06322. Acesso em: out. 2026.

WÖHRER, M.; ZDUN, U. Design patterns for smart contracts in the Ethereum ecosystem. In: 2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData). IEEE, 2018. p. 1513-1520. Disponível em: https://doi.org/10.1109/Cybermatics_2018.2018.00255. Acesso em: out. 2026.

AZIMI, S.; GOLZARI, A.; IVAKI, N.; LARANJEIRO, N. A systematic review on smart contracts security design patterns. Empirical Software Engineering, v. 30, n. 3, art. 95, 2025. Disponível em: https://doi.org/10.1007/s10664-025-10646-w. Acesso em: out. 2026.

SOLIDITY. Solidity Documentation, versão 0.8.19. Disponível em: https://docs.soliditylang.org/en/v0.8.19/. Acesso em: out. 2026.

FOUNDRY. The Foundry Book. Disponível em: https://book.getfoundry.sh/. Acesso em: out. 2026.

OPENZEPPELIN. OpenZeppelin Contracts Documentation. Disponível em: https://docs.openzeppelin.com/contracts. Acesso em: out. 2026.

EIP-1167. Minimal Proxy Contract. Ethereum Improvement Proposals. Disponível em: https://eips.ethereum.org/EIPS/eip-1167. Acesso em: out. 2026.

EIP-1967. Proxy Storage Slots. Ethereum Improvement Proposals. Disponível em: https://eips.ethereum.org/EIPS/eip-1967. Acesso em: out. 2026.

GITCOIN. Allo Protocol. Disponível em: https://www.gitcoin.co/apps/allo-protocol. Acesso em: out. 2026.

ALLO PROTOCOL. Core Allo V2 Contracts. GitHub. Disponível em: https://github.com/allo-protocol/allo-v2. Acesso em: out. 2026.

ALLO PROTOCOL. Allo v2 Documentation. GitHub. Disponível em: https://github.com/allo-protocol/docs. Acesso em: out. 2026.

BATISTA, F. J. S. Allo Protocol V2 (fork refatorado com padrões GoF). GitHub. Disponível em: https://github.com/josiasdev/allo-v2. Acesso em: out. 2026.